

CSE563 — Assignment 5

Unit V: Algorithm Complexity Analysis

Maps to CO5 (determine the algorithmic complexity of various mathematical problems). **Marks: 50.** Attempt all questions. Justify every complexity claim — a bare $O(\cdot)$ answer earns half marks at most.

Part A — Short Answer and Definitions (10 marks, 2 each)

Q1. [2 marks] Define O , Ω and Θ formally. Explain why it is correct but unhelpful to say that binary search is $O(n^2)$.

Solution. $f(n) = O(g(n))$ iff $\exists c > 0, n_0$ such that $f(n) \leq c g(n)$ for all $n \geq n_0$. $f(n) = \Omega(g(n))$ iff $\exists c > 0, n_0$ such that $f(n) \geq c g(n)$ for all $n \geq n_0$. $f(n) = \Theta(g(n))$ iff both hold (with possibly different constants).

Binary search runs in $\Theta(\log n)$, and since $\log n \leq n^2$ for all $n \geq 1$, the statement “binary search is $O(n^2)$ ” satisfies the definition and is **true**. It is unhelpful because O is only an *upper* bound: it says the algorithm is no worse than quadratic, which is a vastly loose claim that fails to distinguish binary search from bubble sort. Use Θ when a tight bound is known.

Q2. [2 marks] Explain the difference between “worst case” and “ O ”. Give an algorithm whose best and worst cases have different orders.

Solution. **Worst case** refers to which *input* we analyse (the input maximising the running time). O refers to how we *bound a function* mathematically. They are independent ideas: one can state an O bound, an Ω bound or a Θ bound on the best-case, average-case or worst-case running time.

Example: insertion sort. Best case (already sorted): each element is compared once, $\Theta(n)$. Worst case (reverse sorted): element i is compared with all i predecessors, $\Theta(n^2)$. So its worst case is $\Theta(n^2)$ and its best case is $\Theta(n)$ — and it is correct to say the algorithm as a whole is $O(n^2)$ and $\Omega(n)$.

Q3. [2 marks] State the three cases of the master theorem. Give one recurrence to which it does not apply and say why.

Solution. For $T(n) = aT(n/b) + f(n)$, let $d = \log_b a$:

1. $f(n) = O(n^{d-\varepsilon})$ for some $\varepsilon > 0 \Rightarrow T(n) = \Theta(n^d)$ (leaves dominate).
2. $f(n) = \Theta(n^d) \Rightarrow T(n) = \Theta(n^d \log n)$ (all levels equal).
3. $f(n) = \Omega(n^{d+\varepsilon})$ and $af(n/b) \leq kf(n)$ for some $k < 1 \Rightarrow T(n) = \Theta(f(n))$ (root dominates).

Does not apply: $T(n) = 2T(n/2) + n \log n$. Here $d = 1$ and $f(n) = n \log n$, which is larger than n^1 but only by a logarithmic factor, not by any polynomial n^ε — it falls in the gap between Cases 2 and 3. (Also inapplicable: $T(n) = T(n-1) + n$, which is subtractive rather than divisive.)

Q4. [2 marks] Define the classes P and NP. What exactly would it take to prove $P = NP$?

Solution. **P**: decision problems solvable by a deterministic algorithm in time $O(n^k)$ for some constant k . **NP**: decision problems for which a proposed *yes*-certificate can be *verified* in polynomial time.

$P \subseteq NP$, since a problem we can solve quickly we can also verify quickly.

To prove $P = NP$ it would suffice to exhibit a **single polynomial-time algorithm for any one NP-complete problem** (say 3-SAT). Because every problem in NP reduces to it in polynomial time, that algorithm would put all of NP into P.

Q5. [2 marks] State the two steps required to prove a problem X is NP-complete, and name the problem proved NP-complete first and by whom.

Solution.

1. Show $X \in \mathbf{NP}$ — give a certificate and a polynomial-time verifier for it.
2. Show X is **NP-hard** — choose a known NP-complete problem Y and give a polynomial-time reduction $Y \leq_p X$ (reduce the known problem *to* the new one, not the other way round).

The first problem proved NP-complete was **SAT** (Boolean satisfiability), by **Stephen Cook** in 1971, independently by **Leonid Levin** — the Cook–Levin theorem. Karp then reduced SAT to 21 further problems in 1972.

Part B — Complexity Analysis and Recurrences (24 marks)

Q6. [5 marks] Determine the Θ complexity of each of the five fragments below, with justification.

(a)

```
1 s = 0
2 for i in range(n):
3     for j in range(n):
4         s += i*j
```

(b)

```
1 i = n
2 while i > 1:
3     i = i // 3
```

(c)

```
1 for i in range(n):
2     j = 1
3     while j < n:
4         j *= 2
```

(d) for i in range(n): for j in range(i, n): <O(1) body>

(e) $T(n) = 4n^3 + 100n^2 \log n + 5^{10}$

Solution. (a) The inner body runs $n \times n$ times: $\Theta(n^2)$.

(b) i takes values $n, n/3, n/9, \dots$ and stops when $n/3^k \leq 1$, i.e. $k \geq \log_3 n$: $\Theta(\log n)$. (The base 3 is only a constant factor versus base 2.)

(c) The outer loop runs n times; the inner doubling loop runs $\log_2 n$ times each and is independent of i : $\Theta(n \log n)$.

(d) The inner loop runs $n - i$ times for $i = 0, \dots, n - 1$:

$$\sum_{i=0}^{n-1} (n - i) = n + (n - 1) + \dots + 1 = \frac{n(n + 1)}{2} = \Theta(n^2).$$

(Half the work of (a), but the same order — constants are discarded.)

(e) 5^{10} is a constant. Comparing n^3 with $n^2 \log n$: $\lim_{n \rightarrow \infty} \frac{n^2 \log n}{n^3} = \lim \frac{\log n}{n} = 0$, so $n^2 \log n = o(n^3)$ and the cubic term dominates: $\Theta(n^3)$.

Q7. [5 marks] Prove from the definition that $4n^2 + 10n + 7 = \Theta(n^2)$, giving explicit constants c_1, c_2, n_0 . Then prove that $2n^3 \neq O(n^2)$.

Solution. Upper bound. For all $n \geq 1$ we have $n \leq n^2$ and $1 \leq n^2$, so

$$4n^2 + 10n + 7 \leq 4n^2 + 10n^2 + 7n^2 = 21n^2.$$

Take $c_2 = 21$, $n_0 = 1$.

Lower bound. All terms are positive, so for all $n \geq 1$:

$$4n^2 + 10n + 7 \geq 4n^2.$$

Take $c_1 = 4$.

Hence with $c_1 = 4$, $c_2 = 21$, $n_0 = 1$ we have $c_1 n^2 \leq 4n^2 + 10n + 7 \leq c_2 n^2$ for all $n \geq n_0$, so the function is $\Theta(n^2)$. ■

$2n^3 \neq O(n^2)$. Suppose it were: then there exist c, n_0 with $2n^3 \leq cn^2$ for all $n \geq n_0$. Dividing by $n^2 > 0$ gives $2n \leq c$ for all $n \geq n_0$ — but $2n$ grows without bound and must eventually exceed any fixed constant c (take $n > \max(n_0, c/2)$). Contradiction. ■

Q8. [6 marks] Solve each recurrence, stating the method used. Assume $T(1) = 1$. (a) $T(n) = T(n-1) + n^2$ (b) $T(n) = 2T(n/2) + n^2$ (c) $T(n) = 3T(n/3) + n$ (d) $T(n) = T(n/2) + n$ (e) $T(n) = 4T(n/2) + n^2$ (f) $T(n) = 2T(n-1) + 1$

Solution. (a) Iteration. $T(n) = n^2 + (n-1)^2 + \dots + 1 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$.

(b) Master, $a = 2$, $b = 2$, $d = \log_2 2 = 1$. $f(n) = n^2 = \Omega(n^{1+1})$; regularity: $2(n/2)^2 = n^2/2 \leq \frac{1}{2}n^2$ with $k = \frac{1}{2} < 1$ ✓. **Case 3:** $\Theta(n^2)$.

(c) Master, $a = 3$, $b = 3$, $d = \log_3 3 = 1$. $f(n) = n = \Theta(n^1)$. **Case 2:** $\Theta(n \log n)$.

(d) Master, $a = 1$, $b = 2$, $d = \log_2 1 = 0$. $f(n) = n = \Omega(n^{0+1})$; regularity: $1 \cdot (n/2) = \frac{1}{2}n$ ✓. **Case 3:** $\Theta(n)$. (Intuition: the work halves each level, $n + n/2 + n/4 + \dots < 2n$ — a geometric series dominated by its first term.)

(e) Master, $a = 4$, $b = 2$, $d = \log_2 4 = 2$. $f(n) = n^2 = \Theta(n^2)$. **Case 2:** $\Theta(n^2 \log n)$.

(f) Iteration. $T(n) = 2T(n-1) + 1 = 4T(n-2) + 2 + 1 = \dots = 2^{n-1}T(1) + (2^{n-1} - 1) = 2^n - 1 = \Theta(2^n)$ (Tower of Hanoi).

Q9. [4 marks] Solve the linear recurrence $a_n = 6a_{n-1} - 8a_{n-2}$ with $a_0 = 2$, $a_1 = 10$, using the characteristic-equation method. Verify a_2 two ways.

Solution. Characteristic equation: $r^2 - 6r + 8 = 0 \Rightarrow (r-2)(r-4) = 0$, roots $r_1 = 2$, $r_2 = 4$ (distinct).

General solution: $a_n = \alpha 2^n + \beta 4^n$.

Initial conditions:

$$n = 0: \quad \alpha + \beta = 2$$

$$n = 1: \quad 2\alpha + 4\beta = 10 \Rightarrow \alpha + 2\beta = 5$$

Subtracting: $\beta = 3$, hence $\alpha = -1$.

$$a_n = 3 \cdot 4^n - 2^n$$

Verification of a_2 . From the closed form: $a_2 = 3(16) - 4 = 44$. From the recurrence: $a_2 = 6a_1 - 8a_0 = 6(10) - 8(2) = 60 - 16 = 44$. ✓

Growth: $a_n = \Theta(4^n)$, governed by the larger root.

Q10. [4 marks] Use a recursion tree to solve $T(n) = 2T(n/2) + n$, then prove your answer by the

substitution method (induction).

Solution. Recursion tree. At level i there are 2^i nodes, each doing $n/2^i$ work, so each level costs $2^i \times n/2^i = n$. The subproblem size reaches 1 after $\log_2 n$ halvings, giving $\log_2 n + 1$ levels.

$$T(n) = \sum_{i=0}^{\log_2 n} n = n(\log_2 n + 1) = \Theta(n \log n).$$

Substitution proof (upper bound). Claim: $T(n) \leq cn \log_2 n$ for $n \geq 2$ and a suitable c .

Basis: $n = 2$: $T(2) = 2T(1) + 2 = 4$, and $c \cdot 2 \log_2 2 = 2c \geq 4$ whenever $c \geq 2$.

Inductive step: assume the bound for all sizes below n . Then

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &\leq 2 \left(c \frac{n}{2} \log_2 \frac{n}{2} \right) + n \\ &= cn(\log_2 n - 1) + n \\ &= cn \log_2 n - cn + n \\ &\leq cn \log_2 n \quad \text{provided } c \geq 1. \end{aligned}$$

Taking $c = 2$ satisfies both requirements, so $T(n) = O(n \log n)$. The matching $\Omega(n \log n)$ follows the same way with the inequality reversed, hence $\mathbf{T}(n) = \Theta(n \log n)$. ■

Part C — Complexity Classes, SAT, Searching and Sorting (16 marks)

Q11. [5 marks] Consider $\Phi = (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge (x_2 \vee x_3)$. (a) Determine whether Φ is satisfiable; if so give a satisfying assignment and verify it clause by clause. (b) How many assignments would exhaustive search examine in the worst case for n variables? (c) Name the class Φ 's decision problem belongs to, given that every clause has at most 2 literals.

Solution. (a) Try $x_1 = \text{T}$. Clause 2 ($\neg x_1 \vee x_3$) then forces $x_3 = \text{T}$. Clause 3 ($\neg x_2 \vee \neg x_3$) with $x_3 = \text{T}$ forces $x_2 = \text{F}$. Check all four clauses with $(x_1, x_2, x_3) = (\text{T}, \text{F}, \text{T})$:

Clause	Evaluation	Value
$(x_1 \vee \neg x_2)$	$\text{T} \vee \text{T}$	T
$(\neg x_1 \vee x_3)$	$\text{F} \vee \text{T}$	T
$(\neg x_2 \vee \neg x_3)$	$\text{T} \vee \text{F}$	T
$(x_2 \vee x_3)$	$\text{F} \vee \text{T}$	T

All clauses true, so Φ is **satisfiable** with $(x_1, x_2, x_3) = (\text{T}, \text{F}, \text{T})$.

(b) Each variable is independently true or false, so there are 2^n assignments; checking each against m clauses costs $O(m)$, for $O(m2^n)$ overall.

(c) Every clause has at most two literals, so this is an instance of **2-SAT**, which is in **P** — solvable in $O(n + m)$ by building the implication graph and checking that no variable shares a strongly connected component with its negation. (General SAT and 3-SAT are NP-complete; the two-literal restriction is what makes it tractable.)

Q12. [5 marks] $A = [4, 8, 15, 16, 23, 42, 57, 61, 78]$ (sorted, indices 0–8). (a) Trace binary search for the key 23, showing lo, hi, mid and the comparison at each step. (b) Trace binary search for the key 30 and state the result. (c) State the maximum number of comparisons for $n = 9$ and for $n = 10^6$, and justify with the decision-tree lower bound.

Solution. (a) **Search for 23.**

lo	hi	mid	$A[\text{mid}]$	Action
0	8	4	23	equal — found at index 4

One comparison only, since 23 is the median element.

(b) **Search for 30.**

lo	hi	mid	$A[\text{mid}]$	Action
0	8	4	23	$23 < 30 \Rightarrow \text{lo} = 5$
5	8	6	57	$57 > 30 \Rightarrow \text{hi} = 5$
5	5	5	42	$42 > 30 \Rightarrow \text{hi} = 4$
lo = 5 > hi = 4: loop ends, return -1 (not found)				

Three comparisons.

(c) The maximum number of comparisons is $\lfloor \log_2 n \rfloor + 1$: for $n = 9$, $\lfloor 3.17 \rfloor + 1 = 4$; for $n = 10^6$, $\lfloor 19.93 \rfloor + 1 = 20$.

Lower-bound justification: model any comparison-based search as a binary decision tree. Each internal node is one three-way comparison and each of the n array positions (plus the “absent” outcomes) must appear as a leaf. A binary tree with n leaves has height at least $\log_2 n$, so *every* comparison-based search needs $\Omega(\log n)$ comparisons in the worst case. Binary search attains this, so it is asymptotically optimal.

Q13. [6 marks] (a) Trace one Lomuto partition of $A = [6, 3, 9, 2, 8, 5]$ with the last element as pivot, showing the array after each swap and the final pivot index. (b) State the best- and worst-case complexity of quicksort and describe the input that triggers the worst case here. (c) Explain why comparison-based sorting cannot beat $\Omega(n \log n)$, and name one sorting algorithm that does run in linear time, stating the assumption that permits it.

Solution. (a) **Partition, pivot = 5 (last element).** i tracks the end of the “ $\leq$ pivot” region, initially $i = -1$; j scans indices 0–4.

j	$A[j]$	Test $A[j] \leq 5$	Array after
0	6	no	$[6, 3, 9, 2, 8, 5]$
1	3	yes: $i=0$, swap $A[0] \leftrightarrow A[1]$	$[3, 6, 9, 2, 8, 5]$
2	9	no	$[3, 6, 9, 2, 8, 5]$
3	2	yes: $i=1$, swap $A[1] \leftrightarrow A[3]$	$[3, 2, 9, 6, 8, 5]$
4	8	no	$[3, 2, 9, 6, 8, 5]$
end	—	swap pivot into $A[i+1] = A[2]$	$[3, 2, \mathbf{5}, 6, 8, 9]$

Final pivot index: 2. Everything left of it (3, 2) is ≤ 5 ; everything right (6, 8, 9) is > 5 . Quicksort now recurses on $[3, 2]$ and $[6, 8, 9]$.

(b) Best case: the pivot splits the array in half every time, $T(n) = 2T(n/2) + n = \Theta(n \log n)$. Worst case: the pivot is always the smallest or largest element, so one side is empty: $T(n) = T(n-1) + n = \Theta(n^2)$. With the *last element* as pivot, the worst case is triggered by an **already-sorted (or reverse-sorted) array** — e.g. $[2, 3, 5, 6, 8, 9]$, where every pivot is the maximum of its subarray. Randomising the pivot or using median-of-three makes the expected time $\Theta(n \log n)$ on every input.

(c) A comparison sort is a binary decision tree whose leaves are the possible output permutations. There are $n!$ permutations, so the tree has $\geq n!$ leaves and height

$$h \geq \log_2(n!) \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n),$$

and the height is the worst-case number of comparisons. Hence no comparison sort can do better.

Linear-time example: counting sort, $\Theta(n + k)$. The assumption that permits it is that the keys are **integers drawn from a bounded range** $[0, k]$ with $k = O(n)$ — it indexes an array

by key value rather than comparing keys, so the decision-tree argument does not apply. (Radix sort extends this to d -digit keys in $\Theta(d(n + k))$.)